ERA · AI CLINIC ASSISTANT

# Era — Complete Documentation

PRD · Architecture · Rules · Design · Task Tracker · Project Memory

v4.3 · compiled October 5, 2026

# Contents


1. Prd.md — Era: AI Face & Voice Clinic Assistant
    - 1. Overview
    - 2. Goals
    - 2b. RMS integration requirements (v3.0)
    - 3. Non-goals (v1)
    - 4. User stories
    - 5. Functional requirements
    - 6. Acceptance criteria
    - 7. Success metrics
2. Architecture.md — Era
    - 1. Shape
    - 2. Runtime data flow (v3.5 — voice-only character)
    - 3. The database layer (v2.0)
    - 4. Client persistence (v3.5)
    - 5. Browser APIs used
    - 5b. RMS integration layer (v3.0)
    - 6. Swap path upward
3. Rules.md — Era project conventions
    - 1. Hard rules
    - 2. Code style
    - 3. Data rules
    - 4. Retrieval rules (`scoreRows`)
    - 5. Browser support & a11y
    - 6. Verification before done
    - 7. Docs
4. Design.md — Era UI
    - 1. Design language
    - 2. Tokens (`style.css :root`)
    - 3. Layout
    - 4. The character (photo, v3.7.2)
    - 5. Replies (voice-only)
    - 6. Components
    - 7. Accessibility & motion
    - 8. Voice UX rules
5. Task.md — Era work tracker
    - ✅ Shipped
    - SOON Next up
    - 🧭 Later
    - 🐞 Known issues
6. Memory.md — Era project memory
    - 1. What this project is
    - 2. Key decisions & why

- 2b. v2.0 — SQLite + API (what changed & why)
- 2c. v3.0 — RMS-ready security layer (what changed & why)
- 2d. v3.1 — Full RMS spec pass (key notes)
- 3. Gotchas / what breaks it
- 4. Environment notes
- 5. If resuming mid-task

# Prd.md — Era: AI Face & Voice Clinic Assistant

**Version:** 3.9.0 (NVIDIA-only voice) · **Status:** Live · **Last updated:** 2026-10-01

## 1. Overview

Era is a browser-based **character** assistant for a doctor's clinic: a 3D-look (Pixar-style) animated clinician who pops up the moment a doctor signs in, greets them by name, and **speaks every reply out loud with lip sync**. She accepts spoken questions (mic) or typed input (slim fallback line), and answers from **two knowledge sources**:

1. **The doctor's dashboard data** — appointments, patients, prescriptions, working hours, scoped to the authenticated doctor through the RMS tool layer (`/api/assistant/chat`).
2. **General health guidance** — a 15-row AI-doctor knowledge base served from **SQLite** (`db/era.db`) through the chat brain (with `/api/qa` as the raw fallback API), covering fever, cold, headaches, blood pressure, diabetes, hydration, sleep, exercise, diet, stress, antibiotics, vaccines, back pain, allergies, and emergency warning signs.

Everything runs locally: static SPA + a zero-dependency Node server with SQLite, with **JWT-based, role-aware, doctor-isolated assistant endpoints** ready to embed in an existing RMS dashboard (no dashboard is built here — Era is the assistant).

## 2. Goals

- Let clinic staff ask about their day by voice, hands-free.
- Give instant first-aid / lifestyle guidance from a curated local knowledge base.
- Stay a **single static folder** — open and run, zero install.

## 2b. RMS integration requirements (v3.0)

- The assistant always operates under the **authenticated** doctor's identity; the frontend never supplies or modifies `doctor_id`, and AI/tool arguments can never widen scope.
- Role-aware behavior: `doctor` (own data only), `admin` (explicit cross-doctor scope only), `reception` (health Q&A only).
- Tools for read **and** write (book/reschedule/cancel) with ownership checks, conflict checks, confirmation-worthy effects, and audit logging.
- Safe errors and data minimization in every response.

## 3. Non-goals (v1)

- Real patient records, prescriptions, or PHI storage.
- Diagnosis. Era gives general guidance and always points to a doctor for urgent signs.
- Multi-day calendar; the model is "today only".

## 4. User stories

| # | As a doctor I want… | So that… | Status |
|---|---|---|---|
| U1 | a friendly face that greets me and speaks | it feels like an assistant, not a form | ✅ |

| # | As a doctor I want… | So that… | Status |
|---|---|---|---|
| U2 | to ask "what's next" / "how many appointments" by voice | I can plan between patients | ✅ |
| U3 | to add a meeting by saying "add a meeting called Review at 3 pm" | I don't stop to type | ✅ |
| U4 | to ask health questions ("what to do for a fever") | I get consistent, safe guidance from the DB | ✅ v1.1 |
| U5 | my name, voice, speed, schedule saved | Era personalizes without re-setup | ✅ |

## 5. Functional requirements

- **FR1 Start flow (v3.5):** after login the workspace appears with **Era popped up**: a personalized spoken greeting ("Good morning, Dr. Amit! I'm Era, your assistant. How can I assist you today?") — once per login session; refreshes restore quietly.
- **FR2 Conversation (v3.5):** voice-first — mic input, spoken replies; one slim text input remains as fallback. No chat log; replies are voice-only with an `aria-live` region for accessibility.
- **FR3 Voice & lip sync:** mic button (Web Speech API, Chrome/Edge); TTS with voice/rate/pitch controls; 3D-look SVG character lip-syncs via viseme mouth shapes (clip rig with teeth + tongue) driven by TTS word-boundary events (+ rAF jaw loop fallback); idle blinks and brow movement keep her alive.
- **FR3b Photo mode (v3.7):** drop a character render at `assets/era3d.png` and it replaces the SVG rig automatically (same float/breathe/talk-pulse motion); missing file falls back to the SVG.
- **FR3c Voice (v4.0):** Era speaks ONLY in **NVIDIA Sofia's voice** (user request: one voice, female, no personas) through the server proxy `POST /api/tts` — `Magpie-Multilingual.EN-US.Sofia` (default) and `Magpie-Multilingual.HI-IN.Sofia`. Per-reply auto-language: Devanagari or romanised Hinglish → हिन्दी Sofia; everything else → English Sofia (Settings can pin हिन्दी). Browser speechSynthesis stays removed — if the voice service is unreachable, Era shows a status message instead of speaking. The API key stays server-side (`nvcf.key`, gitignored). Mic input still uses browser SpeechRecognition.
- **FR4 Schedule brain:** count/list meetings & appointments, next upcoming item (vs current clock), add-by-voice.
- **FR5 Health brain (v2.0):** free-text questions hit `/api/qa` (SQLite `ai_doctor` table); the browser falls back to bundled rows when the API is unreachable; schedule intents still win on overlap.
- **FR6 Settings dialog:** name, Era-speaks toggle, greet toggle, voice select, speed, pitch, schedule editor (`HH:MM | type | title` per line), clear chat history.
- **FR7 Persistence:** settings, schedule, and chat (last 200) in localStorage; knowledge base in SQLite (`db/era.db`), seeded on first server start from `db/db.js`.

## 6. Acceptance criteria

- Asking "i have a fever" returns the fever guidance from the database, spoken aloud.
- Asking "what is the weather" does **not** falsely match any health entry (word-boundary matching).
- "how many appointments today" returns the count and list — never a health answer.
- Deleting `db/era.db` (or localStorage) restores seeds; `node server.js` recreates both.
- App works offline except fonts (graceful fallback).

## 7. Success metrics

- First-response latency < 500 ms for local answers (SQLite lookup); < 1.5 s through the API.
- Zero false-positive health answers on schedule queries (keyword-boundary test suite in repo history).
- 15/15 knowledge rows reachable through natural phrasing.

# Architecture.md — Era

**Version:** 3.9.0 (NVIDIA-only voice) · **Last updated:** 2026-10-01

## 1. Shape

Static SPA + tiny local API server, **zero npm dependencies**. One page, three browser scripts, one Node server. The AI-doctor knowledge base lives in **SQLite** ( `db/era.db` ), served over `/api/qa` ; the browser keeps an offline fallback so the app never breaks.

```
era-assistant/
├── index.html        # single page: auth card, character photo stage, settings <dialog>
├── nvcf.key          # NVIDIA NVCF API key (gitignored — never committed, never logged)
├── style.css         # all styling + keyframe animations (float, breathe, glow, listen, blink)
├── app.js            # session, greeting (once per login session), TTS/ASR glue, lip-sync rig
├── server.js         # Node server: SQLite setup + /api/health + /api/qa + static files
├── package.json      # pins CommonJS (guards against ancestor "type":"module"); npm start
├── db/
│   ├── db.js         # dual-export module (browser global + Node): seed, scoring, ask(), fallback
│   └── era.db        # SQLite database (generated on first server start; git-ignorable)
├── assets/era.png    # avatar
├── Prd.md  Architecture.md  Rules.md  Design.md  Task.md  Memory.md  README.md
└── era-server.log    # server output (git-ignorable)
```

## 2. Runtime data flow (v3.5 — voice-only character)

```
login (auth card) → showApp()
        │
        ├ sessionStorage.era_greeted? — no —➤ personalized greeting → speak()
        │     "Good morning, Dr. Amit! … How can I assist you today?"
        └ yes → quiet restore (character idles; no re-greeting on refresh)
doctor input (mic | slim text fallback)
        │
        ▼
   ask(text) → POST /api/assistant/chat {message}
        │
        ▼
   server: routeIntent → tools (scoped by req.user.doctorId) → reply + tool_result
        │
        ▼
   speak(reply) —➤ POST /api/tts (JWT) → server proxy → NVIDIA NVCF synthesize → WAV → <audio>
        │   voice: S.ttsVoice (Settings dropdown; default Magpie-Multilingual.EN-US.Sofia)
        └ on failure → status message (v3.9: NO browser-voice fallback; speechSynthesis removed)
   (stopSpeak() cancels playback on stop/new reply/logout; the API key stays server-side in nvcf.key)
```

The browser has **no local brain**: dashboard questions are answered only by the doctor-scoped server tools, and identity never comes from the client. ( `/api/qa` remains available as a fallback API.)

## 3. The database layer (v2.0)

**Engine:** SQLite via `node:sqlite` ( `DatabaseSync` ) — built into Node ≥ 22.5, no driver package. **File:** `db/era.db` , created and seeded on server start. Expect one `ExperimentalWarning` line.

| Table | Rows | Schema |
|---|---|---|
| `ai_doctor` | 15 | `id INTEGER PK, q TEXT, keywords TEXT (JSON array), a TEXT` |
| `schedule` | 4 seed | `id INTEGER PK AUTOINCREMENT, time TEXT 'HH:MM', type TEXT CHECK meeting\|appointment, title TEXT` |

**Seeding is idempotent:** `ai_doctor` rows are upserted from `db/db.js` on every start (file is the source of truth); `schedule` inserts only when the table is empty (users own that data). **API:** `GET /api/health` → row counts + version · `GET /api/qa?q=…` → scored best match ( `{found, id, question, answer, score, source:'sqlite'}` or `{found:false}` ). **Scoring ( `scoreRows` , shared browser/server):** phrases substring-match +2; single words match exact token or ≥4-char prefix +1; keywords stored as a JSON string in SQL, plain array in the seed — the parser tolerates both. Highest score wins; 0 → miss.

## 4. Client persistence (v3.5)

`localStorage` ( `era_settings` ) holds voice settings only — the chat log and schedule editor left with the chat panel. `sessionStorage` ( `era_greeted` ) guards the once-per-login-session greeting and is cleared on sign-out. Clearing storage = reset to defaults.

## 5. Browser APIs used

| API | Purpose | Guard |
|---|---|---|
| Web Speech `SpeechRecognition` | mic input | feature-detected; message + typed fallback |
| `speechSynthesis` | voice out | feature-detected; toggle in settings |
| `fetch('/api/qa')` | health answers | try/catch → offline fallback |
| `localStorage` | persistence | try/catch wrapper ( `load` ) |
| `<dialog>` | settings modal | native element |

## 5b. RMS integration layer (v3.0)

Implemented per the RMS assistant spec — the assistant is embeddable in an existing dashboard; **no dashboard is built here**.

```
RMS Login → JWT/session → requireAuth → requireRole/requireDoctor → /api/assistant/chat
  → routeIntent (pluggable "AI") → tools layer (each re-validates ownership)
  → parameterized SQL scoped to req.user.doctorId → sanitize() → reply + card data → dashboard
```

- **Identity:** `verifyJWT()` (HS256, `Authorization: Bearer` or `era_token` HttpOnly cookie) is the only source of `req.user = {id, doctorId, role}` . Never from body/query/URL/AI args.
- **Authorization:** `requireAuth` / `requireRole` / `requireDoctor` / `scopeDoctor` — a doctor is always scoped to their own `doctorId` ; an admin may pass an explicit `doctor_id` (scoped to exactly that); `reception` gets health Q&A only (403 on doctor-scoped tools).
- **Tools:** `getMyAppointments, getMySchedule, getMyPatients, getAppointmentDetails, getAvailableSlots (+ writes` create/reschedule/cancelAppointment`). All parameterized; writes check patient-linkage/appointment ownership first; 409 on slot clash.

- **Chat flow:** `routeIntent(msg, user)` maps language → tool + args (the "AI" slot; swap for an LLM function-caller without touching auth/tools), then executes **through the same tool layer** as direct calls — one permission path for UI and API.
- **Data minimization:** scoped queries + LIMIT/OFFSET; `sanitize()` strips `doctor_id` / `patient_id` from everything returned to the client/AI.
- **Audit:** `assistant_audit(user_id, doctor_id, action, tool_name, request_id, success, detail)` on every chat/tool/login/denied call. No message bodies, no secrets.
- **Errors:** 401 "Please sign in…", 403/404 "I can only access information available to your RMS account." — never SQL, stack traces, or other doctors' data.
- **Swap-in:** replace `verifyJWT` with the real RMS auth, keep the DB or port the schema (§3), set `CORS` to the dashboard origin, set `JWT_SECRET`, and keep demo-login off (`DEMO_LOGIN=0`).

## 6. Swap path upward

1. **LLM:** the API seam already exists — make `/api/qa` call an LLM (schedule + history in context), keep SQLite keyword scoring as fallback; or have `reply()` try the LLM first.
2. **Bigger DB:** `ai_doctor` maps 1:1 to Postgres (`keywords TEXT[]`); swap `DatabaseSync` for `pg` behind the same endpoints — the client never changes.
3. **Calendar sync:** replace the schedule source with Google Calendar/CalDAV events mapped to `{time, type, title}`.

# Rules.md — Era project conventions

**Version:** 3.9.0 (NVIDIA-only voice) · **Last updated:** 2026-10-01

## 1. Hard rules

1. **Zero npm dependencies.** No frameworks, bundlers, or packages — Node built-ins only (`node:http`, `node:sqlite`, `node:fs`). If a feature can't ship without tooling, escalate first.

1b. **CommonJS server code.** `package.json` pins `"type": "commonjs"` (an ancestor folder ships a `"type": "module"` package.json that would otherwise break `server.js`). Don't remove it.

1. **Single page.** All UI lives in `index.html`; do not add pages or routing.
2. **Privacy (v3.9, user-approved exception):** assistant reply TEXT is synthesized by NVIDIA NVCF cloud TTS (Magpie-Multilingual, via the `/api/tts` proxy) — the ONLY voice path. The API key lives server-side only — `NVIDIA_API_KEY` env or gitignored `nvcf.key` — and is never logged or sent to the browser. Everything else stays local: no analytics, no other third-party calls (fonts CDN allowed with graceful fallback). Mic INPUT still uses the browser's local SpeechRecognition.
3. **Medical safety:** Era never diagnoses. Guidance is generic; every urgent-topic answer must include a "see a doctor / emergency" pointer (see `ai_doctor` rows).

4b. **Security invariants (RMS spec):** identity comes only from `verifyJWT`; every tool re-validates ownership via `scopeDoctor()`; parameterized SQL only; never log message bodies or secrets; never leak SQL/stack traces/internal ids in errors; demo-login is controlled by the `DEMO_LOGIN` switch (`1` = on, `0` = off; on by default).

1. **One brain, async.** All answers flow through `POST /api/assistant/chat` (server-side `routeIntent` → scoped tools). `app.js` never answers dashboard questions locally; callers must `await ask()`. The character speaks the reply — voice-only, no chat panel.

5b. **Character contract (v3.7.2):** the character is `assets/era3d.png` in `#photoWrap`, visible by default (no load-event logic). Motion is CSS-only: float/breathe + `talk` pulse on `[data-state=speaking]`. The old SVG rig was removed; `app.js` rig helpers (visemes/blink/brow) remain but no-op safely — if a mouth/eyes rig is ever reintroduced, re-add the elements before relying on them. Assets are served via the `STATIC` map in `server.js` — new assets must be added there. 5d. **Voice (v3.9, NVIDIA-only):** `speak()` = single path: `POST /api/tts` (multipart upstream, see Memory.md) → WAV → `<audio>`. There is NO browser speechSynthesis fallback — do not reintroduce it; on upstream failure Era shows a status message. `S.ttsVoice` comes from the Settings dropdown (Magpie voice list, grouped by language). `speakSeq` still guards playback — stop/new reply/logout must call `stopSpeak()` or the audio keeps playing. NVCF request shape is fixed (multipart form-data, `language` field); don't "simplify" it to JSON. 5c. **Greeting fires once per login session** (guarded by `sessionStorage.era_greeted`, cleared on logout). Refreshes restore quietly. Don't move the flag to localStorage — it must die with the tab session.

## 2. Code style

- Compact ES2022+ vanilla JS; short helpers (`$`, `load`, `save`, `fmt`); template literals.
- Section comments in `app.js`: `// ---- voice out`, `// ---- brain`, etc. Keep them.
- No semicolon-less lines mid-statement; keep the existing dense one-liner style — match it, don't reformat. Don't introduce a linter/prettier config.
- DOM ids are stable API between HTML and JS (`#log`, `#face[data-state]`, `#sSched`, …). Renaming one means renaming both.

## 3. Data rules

- `db/db.js` **is the source of truth** for `ai_doctor` (15 rows) and default schedule; it seeds SQLite (`db/era.db`) on every server start (QA upserted, schedule only when empty).
- **Server = read-only knowledge.** Add/curate rows in `db/db.js`, not by editing `era.db` by hand.
- SQLite `keywords` are a JSON string; seed rows use a plain array — `scoreRows` accepts both.
- Never read `localStorage` without the `load()` try/catch wrapper (v3.5: only `era_settings` remains client-side; chat history and schedule edits were removed with the chat panel).
- Schedule times are `HH:MM` 24-hour strings; types are only `meeting` | `appointment`.
- Health answers come from the DB only — no ad-hoc health strings inside `reply()`.
- New knowledge rows: append to `doctorQA` with a fresh `id`, lowercase `keywords` including likely phrasings; single-word keywords ≥ 4 chars only (boundary/prefix matching relies on it).

## 4. Retrieval rules (`scoreRows`)

- Schedule intents are matched **before** the health lookup in `reply()`.
- Multi-word keyword = substring match (+2); single word = exact token or ≥4-char prefix (+1).
- Highest score wins; 0 → miss (`found:false` over the API, `null` in the fallback).

## 5. Browser support & a11y

- Target evergreen Chrome/Edge; Safari/Firefox get typed chat (mic button explains why).
- Every new control is keyboard-reachable; interactive elements keep `:focus-visible` outlines.
- Respect `prefers-reduced-motion`; don't add non-essential animation outside existing keyframes.

## 6. Verification before done

- `node --check server.js db/db.js app.js` passes (run per file).
- API smoke test: `/api/health` shows 15/4 rows; `/api/qa?q=fever` hits; `q=weather` misses.
- Fallback smoke test: `EraDB.findAnswerFallback` works in Node without a server.
- `node server.js` serves the app at http://localhost:8000 and the mic works in Chrome/Edge.

## 7. Docs

- `Prd.md`, `Architecture.md`, `Design.md`, `Rules.md`, `Task.md`, `Memory.md` are updated in the same change that alters behavior. Version bump + date at the top of each touched doc.
- README stays the 5-line quickstart; depth belongs in the docs above.

# Design.md — Era UI

**Version:** 2.5.0 (NVIDIA-only voice) · **Last updated:** 2026-10-01

## 1. Design language

Calm clinical palette, soft rounded surfaces (radius 14–28 px), pill buttons, airy spacing. One accent does the work: **teal**. Everything else is quiet blues and greys. Typography: **Manrope** (400/600/800) via Google Fonts, fallback `system-ui, sans-serif`.

## 2. Tokens (`style.css :root`)

| Token | Value | Use |
|---|---|---|
| `--bg` | `#eef5f7` | page background |
| `--ink` | `#123240` | text |
| `--mut` | `#5d7784` | secondary text, timestamps |
| `--teal` | `#1f9d9a` | accent: buttons, bubbles, waves |
| `--blue` | `#7fa8cf` | focus rings, ring gradient |
| `--card` | `#fff` | panels |

## 3. Layout

Single centered character stage (`main`, max 1100 px): one `#face` card (max 460 px wide) holds the whorow (identity + sign out), the SVG character stage, wave bars, status, controls and the slim fallback text input. No chat panel — Era's voice and presence are the interface.

## 4. The character (photo, v3.7.2)

- `assets/era3d.png` (3D character render, transparent background) is the character — visible by default in `#photoWrap` (the v3.6 SVG rig was removed in v3.7.2). Floats (`float`) and breathes (`breathe`); `talk` scale-pulse while speaking. Cover-cropped with `--eraPhotoW/H/X/Y` knobs in `:root` (`50% 12%` default keeps the face in frame).
- **Lip sync:** the mouth is a **clip rig** — `#mouthClipPath` (in `defs`) is swapped through the `MOUTH` viseme set (rest/closed/wide/round/open/ooh) on TTS `onboundary` word events; teeth (`#fdfdfd` bar) and tongue (ellipse) sit inside the clipped `#mouthGrp`. A rAF jaw loop adds continuous motion; `onend` restores the rest shape.
- **Idle life:** randomized blinks (`.eye` scaleY via JS, 0.09 s ease) + occasional brow raises (JS swaps the brow `Q` control point, generic for any geometry). `.eye` needs `transform-box: fill-box` for SVG scaling.
- **Listening:** eyes squint (`.eye` scaleY .55), rings pulse red (`listen`), mic turns `#d64545`.
- **Speaking:** wave bars animate (`bar`, staggered delays).
- **Photo mode (v3.7):** if `assets/era3d.png` loads, `#photoWrap` (clean image, no overlay) replaces the SVG and inherits the same motion — float/breathe idle, `talk` scale pulse while speaking. Missing/broken file keeps the SVG rig (graceful fallback).
- States live on `#face[data-state]`: `idle | listening | speaking`; `#status` text mirrors them.

## 5. Replies (voice-only)

Replies are spoken, not rendered as bubbles. `#live` (`.sr-only`, `aria-live="polite"`) carries the current reply text for screen readers and for when "Era speaks" is off. Chat bubbles and the message-entrance animation were removed with the chat panel in v3.5.

## 6. Components

- **Two-view flow (v3.5):** ① `#auth` — doctor login/signup card only; ② `<main>` — the workspace: Era herself (pop-up greeting on entry, voice loop). `.view` = fullscreen gradient screens; JS `show()` toggles `.hidden`. The "Meet Era" intro card and chat panel were removed in v3.5.
- **Chips**: quiet pill buttons (`#eef5f7` bg, `#cfe0e6` border, 0.85 rem).
- **Settings** `<dialog>`: 24 px radius, width `min(460px, 92vw)`, dark backdrop `#123240aa`; stacked labeled fields: name, "Era speaks replies", "Greet me when I start", voice select, rate/pitch sliders. (Schedule editor and clear-chat-history removed with the chat panel in v3.5.)

## 7. Accessibility & motion

- `:focus-visible` gets a 3 px `--blue` outline everywhere.
- `#live` (`.sr-only`) is `aria-live="polite"` — every Era reply is announced even when TTS is off; mic/settings buttons carry `title` tooltips.
- `prefers-reduced-motion: reduce` kills all animations globally.

## 8. Voice UX rules

- Era speaks only the *reply text*, ONLY in **NVIDIA Sofia's voice** (v4.0: single female voice — all other personas removed) via the `/api/tts` proxy — natural, human-like delivery.
- Settings **NVIDIA Voice** is a 2-option select: "Sofia · English" (default) / "Sofia · हिन्दी". Language itself is auto-picked per reply (Devanagari/romanised Hinglish → हिन्दी Sofia).
- Mic press stops current audio first (no overlap while listening); Stop cancels instantly.
- Voice service unreachable → `#status` shows "Voice service unavailable"; Era stays silent until NVIDIA recovers (retries + WAV cache live server-side).

# Task.md — Era work tracker

**Updated:** 2026-10-01

**Status: v4.2 — modular split (lib/env, auth, database, tools, brain, tts, router; server.js = thin entry) + production posture.**

## ✅ Shipped

- ☑ **v3.9.1 — gRPC bridge (diagnosis via official riva client):** upstream "Triton Mapping failed" windows hit BOTH transports independently — gRPC was healthy while HTTP was down (and vice versa). `/api/tts` now tries 3× HTTP then 2× `python tts-grpc.py` (Riva gRPC client, same function-id, WAV built from raw PCM on stdout), validating RIFF each time. `nvidia-riva-client` installed via `pip --user` (diagnostic/bridge tool only — app itself stays zero-npm-dep). gRPC path confirmed serving real audio during an HTTP-down window.
- ☑ **v3.9 — Voice providers removed (user request):** all `speechSynthesis` machinery deleted — `voices` list, `refreshBrowserVoices`, `pickVoice()`, `speakFallback()` sentence chain, viseme/ jaw-loop lip-sync helpers, `speechSynthesis.cancel()` in every path. `speak()` = single NVCF path (fetch WAV via `/api/tts` → `<audio>`); on failure Era shows a status message instead of speaking. Mic (input) still uses browser SpeechRecognition. Settings save no longer stores rate/pitch.
- ☑ **v3.8.1 — Settings cleanup (user request):** Speed/Pitch sliders removed (NVIDIA voice has no rate/pitch params; they only affected the fallback). Settings Voice is now "NVIDIA Voice" — friendly labels ("Sofia · EN-US", "Mia (calm) · EN-US") grouped by language in `<optgroup>`s: English (EN-US) first, then हिन्दी (HI-IN: Sofia/Leo/Siwei/Pascal), then the rest. Offline default list includes a Hindi voice. rate/pitch stay in S as fallback-only defaults (no UI).
- ☑ **v4.1 — Go-live production posture (audit pass):** gitignored `.env` (PORT/HTTPS_PORT/JWT_SECRET; file wins over the box's junk shell env) — real 64-char secret auto-disables `/api/assistant/demo-login`; optional HTTPS mirror on :8443 from gitignored self-signed `certs/{key,cert}.pem` when present; `POST /api/tts` now REJECTS any voice outside Sofia EN/HI (400 BAD_VOICE — audit proved FR voices still synthesized before); demo "APPOINTMENT …" rows re-dated to today/tomorrow at every start ("my appointments tomorrow" never answers empty again); `start-era.cmd` auto-restart watchdog (CRLF!) + logon Scheduled Task ("Era Assistant"; schtasks needs admin — grant on request).
- ☑ **v3.8 — NVIDIA NVCF voice (user request):** browser voice providers demoted to fallback. New server proxy: `POST /api/tts` (JWT-protected, multipart upstream, WAV out) + `GET /api/tts/voices`. Key stored server-side in gitignored `nvcf.key` (or `NVIDIA_API_KEY` env); never logged or sent to the browser. Default voice `Magpie-Multilingual.EN-US.Sofia`; Settings Voice dropdown lists all NVCF voices. app.js plays the returned WAV via `<audio>`, falls back to the browser sentence chain on failure. NVCF quirks: multipart form-data (not JSON), `language` field name, response is a raw RIFF/WAVE file.
- ☑ **v3.7.2 — SVG model removed (user request):** the character is now solely the photo (`assets/era3d.png`, visible by default — no load-event race, no JS dependency). Rig helpers in `app.js` remain but no-op safely (mouth/eyes/brows element refs are empty). Photo motion lives in CSS (float/breathe + talk-pulse on `[data-state=speaking]`).
- ☑ **v3.7.1 — Photo mode live + voice tune:** `assets/era3d.png` saved (copy of the project's 3D character render — transparent background) so Era now appears as the photo; cover-crop display with `--eraPhotoW/H/X/Y` tuning knobs in `:root`. Voice: per-sentence human rate jitter, Neural voices keep native pitch (slider blends 35%), ~140–260 ms breath pause between sentences, mouth returns to rest during pauses.

- ☑ **v3.7 — Photo mode + human voice (user request):** drop a render at `assets/era3d.png` and it replaces the SVG character (clean image, same float/breathe/talk-pulse motion; SVG fallback when absent; whitelisted in `server.js` STATIC + missing-file guard). Voice: `pickVoice()` prefers Edge Natural/Neural (Aria/Jenny) then Chrome Google voices; replies speak sentence-by-sentence with subtle prosody jitter; `speakSeq` token cancels chains on stop/new reply/logout.
- ☑ **v3.6.1 — Character restyle (user choices: fair skin kept, open long hair, grey+white outfit):** ponytail → center-parted open hair (back mass past shoulders + face-framing locks), blue scrubs → light grey apron with V-neck trim under the white coat. Rig untouched (mouth clip, visemes, blinks).
- ☑ **v3.6 — 3D-look character (user request, image-based):** flat SVG replaced with a Pixar-style character — gradient skin/hair/scrubs/coat, glossy brown eyes with catchlights and lashes, side-swept hair + ponytail, crossed arms, badge + stethoscope, soft blurred shadows. Mouth rebuilt as a clipPath rig (teeth + tongue clipped inside the viseme shape); `MOUTH` viseme paths re-tuned to the new geometry; brow flick made geometry-independent. Zero dependencies kept; greeting/lip-sync/security behavior unchanged.
- ☑ **v3.5 — Character UI (user request):** "Meet Era" intro card and chat panel removed — Era IS the interface now. SVG character rig (blinks, brow flicks, viseme mouth), pop-up personalized greeting once per login session (`sessionStorage.era_greeted`, cleared on logout), TTS word-boundary lip sync with rAF jaw fallback, voice-only replies (+ `aria-live` `#live` for a11y), mic-first input with one slim text fallback. Backend untouched — all answers still flow through `/api/assistant/chat` and the doctor-scoped tools.
- ☑ **v3.1 — Full RMS spec (§1–30):** real login (`POST /api/auth/login`, scrypt-hashed passwords, constant-time compare), `PRAGMA foreign_keys=ON` per connection, `prescriptions` table + `GET /api/prescriptions[/:id]`, REST resources (`GET/POST/PATCH/DELETE /api/appointments`, `/api/schedule`, `/api/patients`) with the `WHERE id=? AND doctor_id=?` ownership pattern everywhere, `doctor_id` from `req.user` only (body/query/AI ignored), spec test accounts (Amit/Rahul/Priya), audit `resource`/`resource_id` columns, DB backup before migration (`db/era.backup-*.db`), **16/16 tests** incl. spec TEST 1–10 over live HTTP.
- ☑ **v1.0 — MVP:** animated face, TTS, mic input, typed chat, schedule brain, settings, localStorage.
- ☑ **v1.1 — Knowledge base:** 15-row AI-doctor Q&A + keyword retrieval (word-boundary scoring).
- ☑ **v2.0 — SQLite + API:** `node:sqlite`, `server.js` (`/api/qa`, `/api/health`), seed-on-start, offline fallback, dual-export `db/db.js`, `package.json` CommonJS pin.
    - ☑ **v3.0 — RMS-ready assistant (the pasted spec, implemented in Era):**
    - JWT (HS256) auth middleware; identity **only** from token/cookie — never body/query/AI args.
    - Roles: `doctor` / `admin` / `reception`; centralized `requireAuth` / `requireRole` / `requireDoctor` / `scopeDoctor`.
    - RMS schema: `users, doctors, patients, doctor_patients, doctor_schedules, appointments, assistant_audit` + FKs, CHECKs, timestamps, indexes on all doctor-scoped fields.
    - Tools: `getMyAppointments, getMySchedule, getMyPatients, getAppointmentDetails, getAvailableSlots` + writes `createAppointment, rescheduleAppointment, cancelAppointment` — each re-validates ownership; parameterized SQL only.
    - `POST /api/assistant/chat`: the spec's 11-step server flow (routeIntent = pluggable "AI"; swap for an LLM without touching auth/tools).
    - Audit log (`assistant_audit`): user/doctor/action/tool/request_id/success; no prompts or secrets stored.
    - Data minimization: scoped queries + LIMIT/OFFSET + `sanitize()` strips `doctor_id`/`patient_id` from anything leaving the server. - Frontend: server-derived identity (`/me`), persona picker + demo-login (v4.3: `DEMO_LOGIN` switch, default ON; `DEMO_LOGIN=0` disables), appointment/schedule/patient/slot cards, permission-denied state, offline fallback.
    - `token.js` CLI to mint test JWTs; `test/security.test.js` — **15/15 pass**, isolated scratch DB per run.

## Next up

- ☑ **v3.4 — Spec gap closure (§7/§8/§13/§14):** `GET /api/appointments/:id`, `GET /api/patients/:id`, prescription create/update/delete (REST), `patient_name` column (migrated additively) accepted on appointment create in REST **and** AI tools; 3 new regression tests → suite now 18/18.
- ☑ **v3.3 — Three-view UI:** ① auth page (login/signup only) → ② "Meet Era" getting-started (avatar, feature steps, Get Started) → ③ workspace (face + chat + voice). Flash-of-content guard, auth page messages, session restore on reload; logout available from workspace and intro.
- ☑ **v3.3 — Appointments panel removed from UI** (user request): chat cards are the only appointment surface in the client; `/api/appointments` + tools remain fully available for the embedding dashboard.
- ☑ **v3.2 — Logout:** `POST /api/auth/logout` clears the HttpOnly cookie (audited); /me afterwards = 401. Stateless-JWT note: instant revocation needs a denylist (backlogged with R5).
- ☐ **R1 LLM behind /chat:** replace `routeIntent` with a function-calling LLM (tools unchanged; prompt ≠ security boundary).
- ☐ **R2 Real RMS wiring:** your login + your DB per README §"Wiring into your real RMS".
- ☐ **R3 Pagination everywhere** ( `getMyPatients` has LIMIT/OFFSET; apply to appointments if lists grow).
- ☐ **R4 Confirmation UX for writes:** show a confirm chip before `createAppointment` / `cancel` fires.
- ☐ **R5 Refresh tokens + logout revocation** (current tokens are 8 h, stateless).

## Later

- ☐ R6 Postgres swap ( `node:sqlite` → `pg` ) for multi-user production.
- ☐ R7 PWA + offline caching; calendar sync; Hindi/regional voice input.

## Known issues

- None open. (Fixed in v3.0: base64url decode bug, JSON replacer bug, scratch-DB test isolation.)

# Memory.md — Era project memory

**Updated:** 2026-10-01 · Read this before touching `app.js` or `db/db.js`.

## 1. What this project is

Single-folder, static, zero-dependency **character** assistant for a clinic ("Era"): inline SVG clinician (v3.5) with lip sync, Web Speech in/out, and dashboard answers from the doctor-scoped server tools. The browser holds no local brain — every answer comes from `/api/assistant/chat`.

## 2. Key decisions & why

| Decision | Why |
|---|---|
| JS-file "database" (`window.EraDB`) instead of real DB | Project is static/dependency-free by rule; JSON-in-JS keeps zero-install, and the table shape maps 1:1 to SQL for the later swap (Architecture.md §5). |
| Keyword scoring, not regex-per-topic | 15 topics × phrasings would explode regexes; scoring gives graceful best-match and a clean `null` fall-through. |
| Health lookup **after** schedule intents | "how many appointments today" contains no health keywords, but a single over-eager keyword (e.g. *weight* in diet row) must never shadow schedule answers; ordering makes intent overlap deterministic. |
| Word-boundary/prefix token match | Earlier `t.includes(k)` matched "w**eat**her" → diet row. Tokens (`w===n` or `w.startsWith(n)` when `n≥4`) killed that class of bug; prefixes catch *vaccine/vaccines*. |
| Schedule stored only in localStorage | User edits must survive reload; `db.js` seeds it on first run (`load(KEY.sch,null) \|\| EraDB.seedSchedule.slice()`). |
| Greeting mentions health skill | Otherwise users never discover the v1.1 feature. |
| `speak()` strips `•` and `–` | Screen-reader-friendly bullets sound broken through TTS. |

### v3.8 — NVIDIA NVCF voice (what changed & why)

| Decision | Why |
|---|---|
| **v3.9.1: BOTH transports flake independently — try HTTP then gRPC** | Same function-id, two serving paths. Verified gRPC healthy during an HTTP-down window (and later the reverse). Server: 3× HTTP → 2× `python tts-grpc.py` (riva gRPC client, `pip install --user nvidia-riva-client`; tool-only dep, app stays zero-npm). gRPC `synthesize()` returns `resp.audio` = raw s16le PCM → bridge builds the 44-byte WAV header itself. gRPC has NO list_voices method (HTTP only). |
| --- | --- |
| **Upstream is FLAKY — design for it** | NVCF's Triton ensemble intermittently returns `400 "Mapping failed. Check that all pronunciations…"` for MINUTES at a time, for ANY text (same text was 200 earlier — it's time-based, not input-based). Defense: server retries 4x (0.8/2/4.5s backoff) + validates `RIFF` header + LRU WAV cache (20) + browser fallback in `speak()`. Do not trust a single success/failure as proof. |
| Server proxy `/api/tts`, not browser→NVIDIA direct | The API key must never reach the browser; also NVCF would CORS-block a direct call. Key in gitignored `nvcf.key` or `NVIDIA_API_KEY` env — never hardcode, never log. |

| Decision | Why |
|---|---|
| NVCF synth is **multipart/form-data**, field is `language` | First JSON attempt returned `400 Bad Request, empty text input`. Schema lives in `{base}/openapi.json` (Riva HTTP API 1.9.0). Response is a raw RIFF/WAVE — play directly in `<audio>`. |
| Voice name embeds the language | `Magpie-Multilingual.EN-US.Sofia` → derive `language=EN-US` from the name (regex on the name). List from `GET /v1/audio/list_voices`. |
| v3.9: browser speechSynthesis REMOVED (user request) | Only Magpie-Multilingual voices now — `speak()` is a single NVCF path; upstream failure → status message, no fallback voices. `stopSpeak()` consolidates the old stop/cancel paths. Mic input is unaffected (SpeechRecognition stays). |
| `speakSeq` token still guards everything | stop/new reply/logout must call `stopSpeak()` — playback and (any future) chains share the token. |
| --- | --- |
| Server proxy `/api/tts`, not browser→NVIDIA direct | The API key must never reach the browser; also NVCF would CORS-block a direct call. Key in gitignored `nvcf.key` or `NVIDIA_API_KEY` env — never hardcode, never log. |
| NVCF synth is **multipart/form-data**, field is `language` | First JSON attempt returned `400 Bad Request, empty text input`. Schema lives in `{base}/openapi.json` (Riva HTTP API 1.9.0). Response is a raw RIFF/WAVE — play directly in `<audio>`. |
| Voice name embeds the language | `Magpie-Multilingual.EN-US.Sofia` → derive `language=en-US` from the name (regex on the name). List from `GET /v1/audio/list_voices`. |
| v3.9: browser speechSynthesis REMOVED (user request) | Only Magpie-Multilingual voices now — `speak()` is a single NVCF path; upstream failure → status message, no fallback voices. `stopSpeak()` consolidates the old stop/cancel paths. Mic input is unaffected (SpeechRecognition stays). |
| `speakSeq` token still guards everything | stop/new reply/logout must call `stopSpeak()` — playback and (any future) chains share the token. |

## v4.0 — Sofia-only voice + auto-language (what changed & why)

| Decision | Why |
|---|---|
| ONE voice: Sofia EN-US (default) + HI-IN only; server filters `/api/tts/voices`, app hard-filters too | User request ("only 1 voice, ladki ki voice"). Sofia exists in EN-US, HI-IN, AR-AR — Arabic dropped per "only hindi english hinglish". |
| Auto-language per reply in `pickVoice()`: Devanagari → हिन्दी, romanised Hinglish word-list → हिन्दी, else selected voice | User wants Hindi/English/Hinglish without picking languages manually. Verified live: roman Hindi text synthesizes fine with HI-IN. |
| `language` field must be proper-case (`en-US`), never voice-name case (`EN-US`) | Uppercase tag → Triton `400 "Mapping failed"` EVERY time (root cause of era-server.log failures). Derive then re-case `xx` + `-` + `YY`. |
| TTS failure log now records EVERY attempt (`errs[]`), not just the last | gRPC error used to overwrite the HTTP errors, hiding the real cause during debugging. |
| Stale `S.ttsVoice` (old localStorage, e.g. Mia) never breaks `pickVoice()` | Falls back to `SOFIA.en`; Settings save normalizes it. |

## v4.1 — Production posture (what changed & why)

| Decision | Why |
|---|---|
| `.env` file WINS over shell env (`process.env[m[1]]=…` unconditionally) | This box exports junk shell-wide (`PORT=0` broke the first launch). One source of truth per process. |
| `POST /api/tts` validates `SOFIA_VOICES` (400 BAD_VOICE), same const as the list endpoint | Audit proved Louise (FR) still synthesized → "Sofia only" was cosmetic. Single owner = list & guard can't drift. |
| HTTPS mirror reuses the shared `handler(req,res)` — NOT the `http.Server` instance | `https.createServer(opts, serverInstance)` throws `listener must be function` at startup (bit us live). |
| `start-era.cmd` MUST be CRLF; launch with an ABSOLUTE path | LF batch files: labels/goto misfire; this sandbox sets `NoDefaultCurrentDirectoryInExePath` so bare `cmd /c start-era.cmd` is "not recognized". |
| Demo appointment rows (reason `APPOINTMENT …`) re-dated to today/tomorrow at every start | Seed dates froze at first DB creation → "appointments tomorrow" answered empty forever after day one. |
| `schtasks` logon-task registration left to the user (elevation denied in-session) | Watchdog still runs manually; README documents the one-line register command. |

## v4.2 — Modular split (what changed & why)

| Decision | Why |
|---|---|
| server.js (531 lines) → 7 modules under `lib/`: env → auth → database → tools → brain → tts → router; server.js = thin entry (config init + listeners + test exports) | Two consecutive design audits capped at 4/10 while every concern lived in one file. Now each concern is readable alone; dependencies flow one way (router → everything; env → everything). |
| `SOFIA_VOICES` has ONE owner (`lib/env.js`); both the voice list and the synthesize 400-guard import it | Policy was stated twice (drift risk). |
| TTS retry/cache/key pipeline extracted to `lib/tts.js` returning `{wav,errs}` | Router now owns only HTTP; fixed the latent `lastErr`-after-`errs` bug in the gRPC catch during the move. |
| `__test` export surface preserved exactly (test/security.test.js, token.js untouched) | Behavior-preserving refactor: same endpoints, same .env/nvcf.key/certs handling, same :8000/:8443. Verified: 18/18 suite, EN/HI/Hinglish 200 RIFF, Louise 400, watchdog restart to new code. |

## v4.3 — Demo login is a switch, not a side effect (what changed & why)

| Decision | Why |
|---|---|
| `DEMO_LOGIN` flag in `lib/env.js` (default ON, `DEMO_LOGIN=0` to disable); router checks the flag, NOT the JWT_SECRET value | v4.1 tied demo-login to "no real secret" — going live silently killed the start screen's advertised demo users (dead end the user hit). Now demo access is an explicit product decision, independent of auth strength. |
| Frontend still advertises the demo users; endpoint is ON by default | The login screen and the endpoint must agree — a screen offering dead buttons is worse than no feature. |
| One-click demo sign-in re-verified in-browser after the un-gate (greeting + TTS + chat) | Contract: "click a demo user → signed in as that doctor". |

### v3.5 — Character UI (what changed & why)

| Decision | Why |
|---|---|
| Greeting guarded by `sessionStorage` (not localStorage) | User chose "once per login session": sessionStorage dies with the tab session, so a fresh login re-greets but a refresh doesn't. Cleared explicitly on logout. |
| Viseme lip sync from TTS `onboundary` word events | Only zero-dep way to approximate phoneme timing; vowel content of each word picks the mouth shape (round/ooh for u/o words, wide/open otherwise). rAF jaw loop covers engines without boundary events (e.g. some Safari voices). |
| No chat panel — voice-only replies + hidden `aria-live` | User explicitly wanted the character, not a chat UI. `#live` keeps replies accessible when TTS is off and satisfies a11y review. |
| SVG replaces `assets/era.png` in the workspace | Lip sync needs an animatable mouth; the PNG can't. The PNG stays on the auth card (logo). |
| v3.6: mouth is a clipPath rig (`#mouthClipPath` in defs) | Pixar-look needs teeth/tongue that stay inside the mouth while the viseme shape morphs — clipping gives that for free. `app.js` swaps the clip path's `d` via the `MOUTH` set; never draw over `#mouthGrp` elsewhere. |
| v3.7: photo mode = `<img>` probe, not a config flag | `assets/era3d.png` loads → `#photoWrap` replaces the SVG; error → SVG stays. No build step, no setting to remember — the user just drops the file (chat-pasted images can't be written to disk by the agent). |
| v3.7: sentence-chained TTS with `speakSeq` token | Per-sentence utterances give natural pacing + allow tiny rate/pitch jitter; the token makes stop/new-reply/logout abort the chain (Web Speech `cancel()` fires `onend`, which would otherwise keep speaking). |
| Server-side reply() kept, local brain removed | All answers must stay doctor-scoped server-side; removes the localStorage schedule mutation path entirely. |

## 2b. v2.0 — SQLite + API (what changed & why)

| Decision | Why |
|---|---|
| `node:sqlite` (`DatabaseSync`), zero deps | A real SQL file DB with no driver install; keeps the no-npm rule. Node ≥ 22.5; one ExperimentalWarning at start is expected. |
| `server.js` owns DB + static files | One process serves app + API; replaced the Python http.server (kill it first — port 8000 clash). |
| API-first with offline fallback (`ask()`) | Browser calls `/api/qa`; on fetch failure uses bundled rows — the app never dies when the server does. |
| Dual-export `db.js` (factory, UMD-style) | Same file feeds the browser (`window.EraDB`) and seeds/queries SQLite in Node (`module.exports`). |
| `package.json` pins `"type":"commonjs"` | The user's HOME folder has a package.json with `"type":"module"`; ancestor resolution broke `server.js` (`require is not defined`). The project-local pin wins — do not delete it. |
| `keywords`: JSON string in SQL, array in seed | SQLite TEXT column; `scoreRows` parses defensively (accepts both). Early bug: `JSON.parse` on the seed array crashed — caught by the Node smoke test. |

## 2c. v3.0 — RMS-ready security layer (what changed & why)

| Decision | Why |
|---|---|
| `verifyJWT` accepts Bearer **or** HttpOnly cookie | Dashboard pages can't always set headers; cookie keeps the token out of JS. |
| `scopeDoctor()` centralizes scoping | Doctors are pinned to their own doctorId; only admin may pass an explicit `doctor_id` — one choke point, no per-tool drift. |
| `routeIntent` separate from tools | The "AI" is replaceable (swap for an LLM) while auth stays in the tool layer — prompt is never a security boundary. |
| `sanitize` strips `doctor_id` / `patient_id` in responses | Data minimization; the client never learns internal keys. |
| Scratch DB via `ERA_DB` env for tests | Write tools mutate state; without isolation, run #2 fails on already-cancelled rows. `server.js` resolves it at require-time — set it **before** requiring. |
| Demo-login disabled by `JWT_SECRET` env | Demo convenience can't leak into production; a real RMS brings its own login. |

### v3.0 bug post-mortems (both caught by verification, not review)

- **JWT verify always failed:** `Buffer.from(s).toString()` decodes UTF-8, not base64url — payload parsed as garbage while the *signature* was valid. Fix: `Buffer.from(s,'base64url').toString('utf8')`.
- **`tool_result` collapsed to `""`:** JSON replacer returned the *key* (`k`) instead of *value* (`v`) when dropping internal ids, stringifying the whole object. Fix: replacer `(k,v)=>…`.

## 2d. v3.1 — Full RMS spec pass (key notes)

- Workspace inspection confirmed **no separate RMS project exists** — Era IS the existing project; the spec was implemented in-place (no rebuild). DB backed up before reseeding.
- Passwords: scrypt (`crypto.scryptSync`, static salt `era-salt-v1`) + `timingSafeEqual`. Demo-grade — swap for bcrypt/argon2 with per-user salts before production.
- Fresh seed required: pre-v3.1 DBs lack `password_hash`; delete `db/era.db` (after backup) to reseed.
- REST + AI tools accept both key styles (`date` / `appointment_date`, `start_time` / `appointment_time`) so the spec's naming and Era's tool naming coexist.
- Test 3 over HTTP returns **404** (not 403) by design: ownership failures are indistinguishable from "doesn't exist" to avoid resource enumeration.

## 3. Gotchas / what breaks it

- **SQLite duplicate-column error text is "duplicate column name: ...", not "already exists"** — the idempotent-migration catch must match both or boot #2 crashes (cost a restart loop in v3.1).
- **Login is sign-in + sign-up combined** (user request): unknown email+password auto-creates an account (201); wrong password on an existing account still 401s with a hint. Don't "restore" a hard register flow.
- **Chat 403 → friendly 200 denial for non-doctor roles:** admin/reception asking for doctor data get `ok:true, denied:true` + guidance text; the tool layer still enforces security. Don't turn this back into a raw 403 — the browser console errors confused the user.

- **Prefix matching ≥ 4 chars only** — shorter prefixes would make `wa` match *walk* and *water*. Keep single-word keywords ≥ 4 chars (Rules.md §3).
- `findAnswer` normalizes away punctuation; keywords are matched lowercase — keep them lowercase.
- v3.5 removed the client `reply()` brain, chat log, and schedule editor — don't reintroduce local intent handling in `app.js`; answers come only from `/api/assistant/chat` (Rules 5).
- v3.6 character geometry: mouth center ≈ (150,166); viseme `MOUTH` paths in `app.js` are tuned to it. If you move the mouth in `index.html`, re-tune `MOUTH` + the teeth/tongue rects inside `#mouthGrp` together — they move as a set. Brows use a generic Q-control-point raise (works for any brow path).
- v3.7 photo mode: `assets/era3d.png` is the character image (v3.7.1: copy of `era.png`, the 3D render with transparent background). v3.7.2: **SVG model deleted** — the photo is the only character, visible by default in HTML (no load-event race). Rig helpers in `app.js` no-op safely (element refs empty). Photo framing knobs: `--eraPhotoW/H/X/Y` in `style.css :root`. New static assets must be added to the `STATIC` map in `server.js`.
- v3.7 voice: `speakSeq` must be bumped in every cancellation path (Stop button, `doLogout`, starting a new reply). Miss one and the sentence chain keeps talking after cancel. Local voices only — cloud TTS would break Rules 3 (privacy/no third-party calls).
- `reply(q)` now keeps the **raw** input in `input` because `q` is lowercased in place — pass `input` (not `q`) to `findAnswer`. (It's case-safe either way, but that's the contract.)
- Deleting localStorage = reset to seed; that's a feature (Rules §3), don't "fix" it.
- Mic on `file://` is blocked in Chrome — always serve via the local server; that's why the README insists on `python -m http.server`.
- `era-server.log` is server output, not project data; safe to delete/git-ignore.

## 4. Environment notes

- Windows + bash (Git Bash): use POSIX syntax; `python` = Python 3.13, `node` = v24.
- Server running on **:8000** is `node server.js` (nohup, log in `era-server.log`); stop old listeners first — port clash. `db/era.db` regenerates if deleted; **test runs never touch it** (they use `db/era.test.db` via `ERA_DB`).
- **Editing `server.js` requires a server restart** — the running process keeps old code; a curl against a stale server will fail even after a correct fix (cost us a debug loop in v3.0).
- Verification loop used: `node --check` → Node smoke test of `findAnswer` (set `global.window={}`, `require('./db/db.js')`) → `curl` the served paths for 200.

## 5. If resuming mid-task

1. Check `Task.md` — shipped vs backlog tells you the frontier.
2. Retrieval changes ⇒ re-run the 7-case smoke test (fever/diet/BP hit; weather/schedule miss).
3. Doc updates travel with code changes (Rules.md §7); bump versions + dates.
4. If a health answer misfires, check keyword collisions **and** intent ordering in `reply()` first.